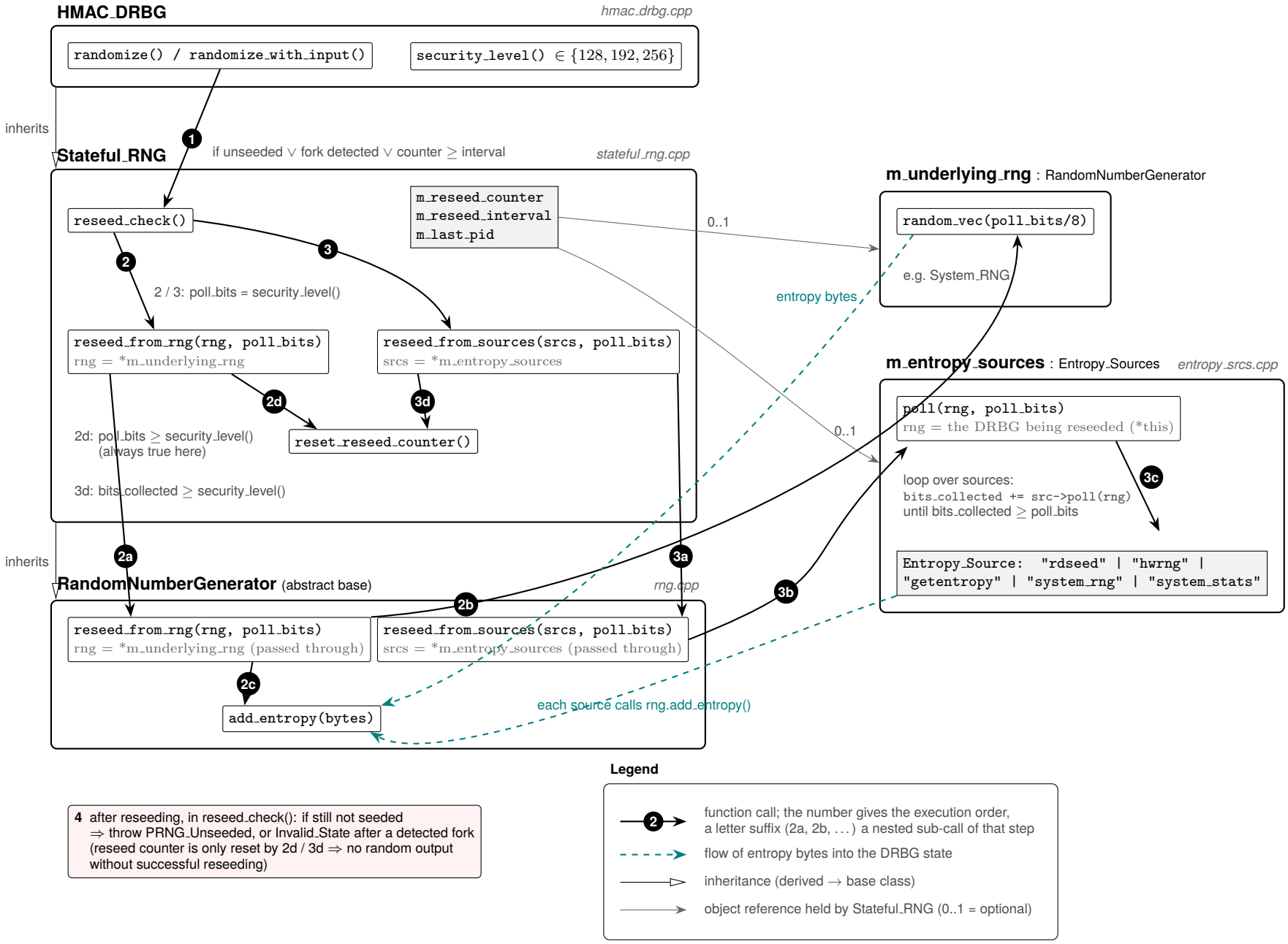
HMAC_DRBG
hmac_drbg.cpp
randomize() / randomize_with_input()
security_level() ∈ {128, 192, 256}
inherits
Stateful_RNG
if unseeded ∨ fork detected ∨ counter ≥ interval
stateful_rng.cpp
reseed_check()
m_reseed_counter
m_reseed_interval
m_last_pid
0..1
m_underlying_rng : RandomNumberGenerator
random_vec(poll_bits/8)
e.g. System_RNG
2 / 3: poll_bits = security_level()
entropy bytes
reseed_from_rng(rng, poll_bits)
rng = *m_underlying_rng
reseed_from_sources(srcs, poll_bits)
srcs = *m_entropy_sources
m_entropy_sources : Entropy_Sources
entropy_srcs.cpp
2d: poll_bits ≥ security_level()
(always true here)
reset_reseed_counter()
poll(rng, poll_bits)
rng = the DRBG being reseeded (*this)
0..1
loop over sources:
bits_collected += src->poll(rng)
until bits_collected ≥ poll_bits
3d: bits_collected ≥ security_level()
Entropy_Source: "rdseed" | "hwrng" | "getentropy" | "system_rng" | "system_stats"
inherits
RandomNumberGenerator (abstract base)
rng.cpp
reseed_from_rng(rng, poll_bits)
rng = *m_underlying_rng (passed through)
reseed_from_sources(srcs, poll_bits)
srcs = *m_entropy_sources (passed through)
add_entropy(bytes)
each source calls rng.add_entropy()
4 after reseeding, in reseed_check(): if still not seeded
⇒ throw PRNG_Unseeded, or Invalid_State after a detected fork
(reseed counter is only reset by 2d / 3d ⇒ no random output
without successful reseeding)
Legend
function call; the number gives the execution order,
a letter suffix (2a, 2b, . . .) a nested sub-call of that step
flow of entropy bytes into the DRBG state
inheritance (derived → base class)
object reference held by Stateful_RNG (0..1 = optional)